



**UNIVERSIDADE FEDERAL DE RORAIMA
CENTRO DE CIÊNCIA E TECNOLOGIA
DEPARTAMENTO DE CIÊNCIA DA COMPUTAÇÃO**



DCC301– ARQUITETURA E ORGANIZAÇÃO DE COMPUTADORES – 2026.2

PROF. DR. HERBERT OLIVEIRA ROCHA

**AURÉLIO ARENDARTCHUK JÚNIOR
JOÃO GUILHERME BARROS JÁCOME**

[Link do Repositório](#)

**LABORATÓRIO DE CIRCUITOS
Subsistema de Memória e Unidade de Controle**

BOA VISTA, RR

2026

AURÉLIO ARENDARTCHUK JÚNIOR
JOÃO GUILHERME BARROS JÁCOME

LABORATÓRIO DE CIRCUITOS
Subsistema de Memória e Unidade de Controle

Relatório do Laboratório de Circuitos,
Projeto integrador sobre Subsistema
de Memória e Unidade de Controle da
disciplina de Arquitetura e
Organização de Computadores.

Docente: Prof. Herbert Oliveira Rocha

BOA VISTA, RR

2026

SUMÁRIO

1. INTRODUÇÃO.....	4
2. Mapa de memória e decodificação de endereços.....	4
2.1 A cache: organização e divisão do endereço:.....	8
2.2 Traço obrigatório de 16 acessos.....	12
2.3 Teste de conflito com o detector de números primos.....	13
7. CONCLUSÃO.....	17
8. Declaração de uso de ferramentas de IA generativa.....	17
9. REFERÊNCIAS.....	17

1. INTRODUÇÃO

Este relatório documenta o desenvolvimento do Laboratório de Circuitos. O trabalho que foi dividido em duas partes de peso igual foi dividido individualmente para cada integrante, sendo unificados num mesmo relatório, do qual está organizado em duas grandes seções, espelhando a estrutura do enunciado.

A seção da Parte I descreve o mapa de memória e a decodificação de endereços, o mecanismo de paridade da RAM, a organização da cache e a forma como o endereço de 6 bits é reinterpretado em rótulo, índice e deslocamento, os resultados do traço obrigatório de 16 acessos e o teste de conflito construído a partir de um detector de números primos, evidenciando a principal limitação de uma cache de mapeamento direto.

A seção da Parte II apresenta o diagrama de estados da unidade de controle, a tabela de transição, os mapas de Karnaugh e as equações booleanas resultantes para a versão cabeada, o formato da microinstrução e o conteúdo da ROM de microcódigo para a versão microprogramada, e uma comparação quantitativa entre as duas implementações.

2. Mapa de memória e decodificação de endereços

A técnica de decodificação parcial de endereços utilizada, onde apenas os bits mais significativos do barramento (A5, A4) são decodificados para selecionar o dispositivo, enquanto os bits restantes endereçam livremente dentro dele. Cada dispositivo responde a uma faixa (região) do espaço de endereçamento, e um decodificador de endereços centraliza a arbitragem de qual dispositivo está "ativo" a cada instante, evitando que dois dispositivos respondam simultaneamente ao mesmo endereço.

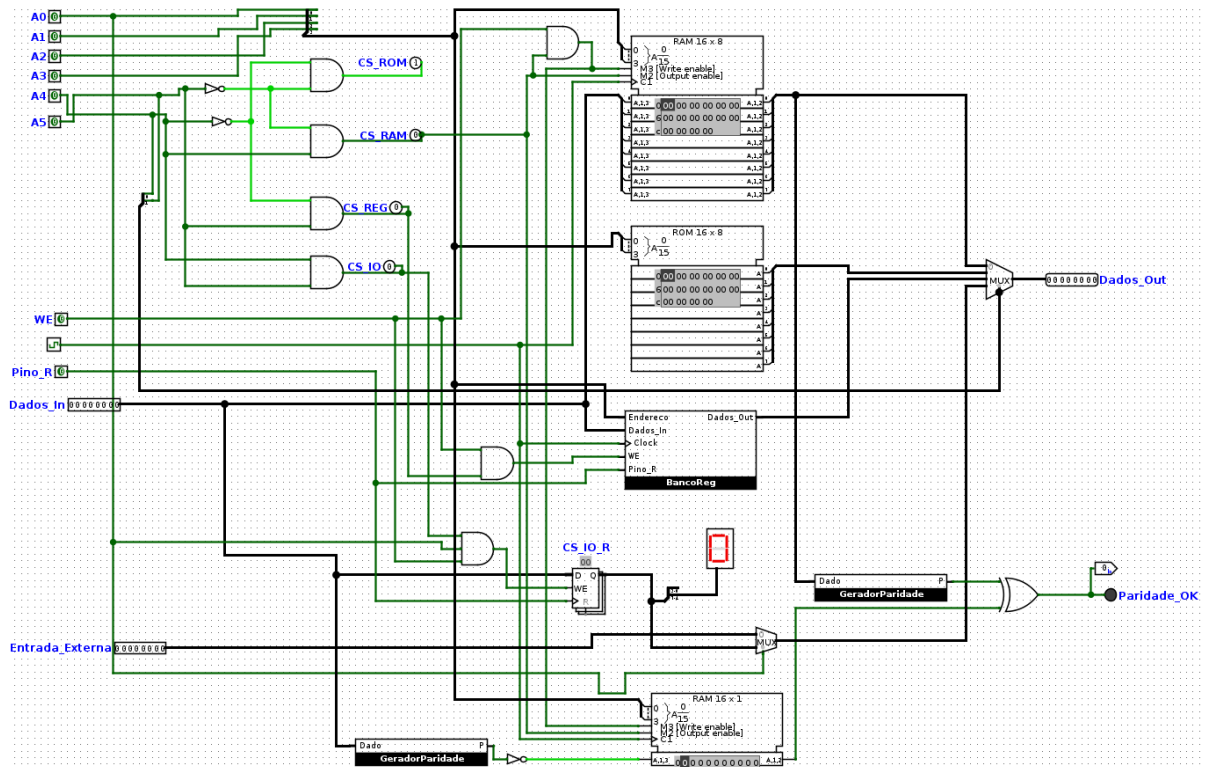


Figura 1 - Visão Geral da Memória

O módulo de memória possui um barramento de endereços de 6 bits, endereçando 64 posições. Os dois bits mais significativos (A5, A4) alimentam um decodificador 2→4 que seleciona qual dos quatro dispositivos responde a cada endereço; os quatro bits restantes (A3..A0) endereçam uma das 16 posições internas do dispositivo selecionado.

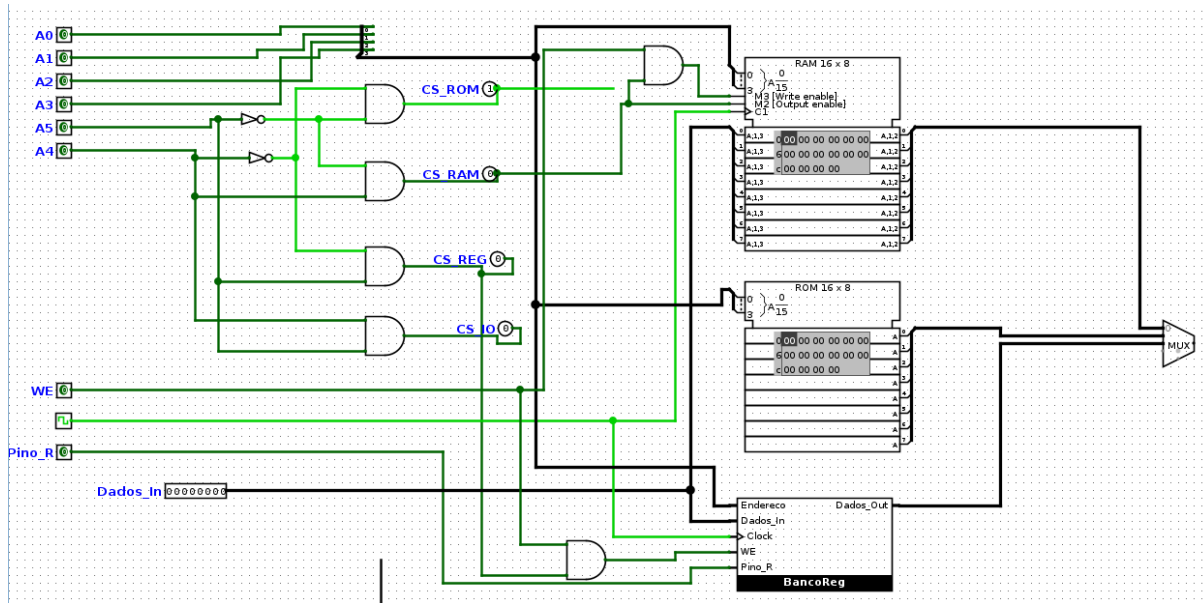


Figura 2 - Representação A0 - A5

Faixa	Dispositivo	Sinal de seleção
0x00–0x0F	ROM de programa	$CS_ROM = A5' \cdot A4'$
0x10–0x1F	RAM de dados	$CS_RAM = A5' \cdot A4$
0x20–0x2F	Banco de registradores	$CS_REG = A5 \cdot A4'$
0x30–0x3F	Região de E/S	$CS_IO = A5 \cdot A4$

Tabela 1 - Endereçamento do A4 - A5

Dessa forma podemos obter a tabela-verdade dos sinais de seleção que ficou dessa forma:

A5	A4	CS_ROM	CS_RAM	CS_REG	CS_IO
0	0	1	0	0	0
1	0	0	0	1	0
0	1	0	1	0	0

1	1	0	0	0	1
---	---	---	---	---	---

Tabela 2 - Tabela-verdade dos sinais de seleção A4 - A5

O circuito foi implementado com 2 portas NOT (gerando A5' e A4') e 4 portas AND de 2 entradas, uma por equação (Como pode ser observado na Figura 1). A seleção do dado de saída entre os quatro dispositivos é feita por um multiplexador de 4 entradas (Multiplexador 02), comandado diretamente pelos mesmos bits A5, A4, desta forma, foi dispensado a necessidade de ligar fisicamente cada CS a um dispositivo que não possua pino de habilitação (por exemplo a ROM, que só possui Endereço e Dado).

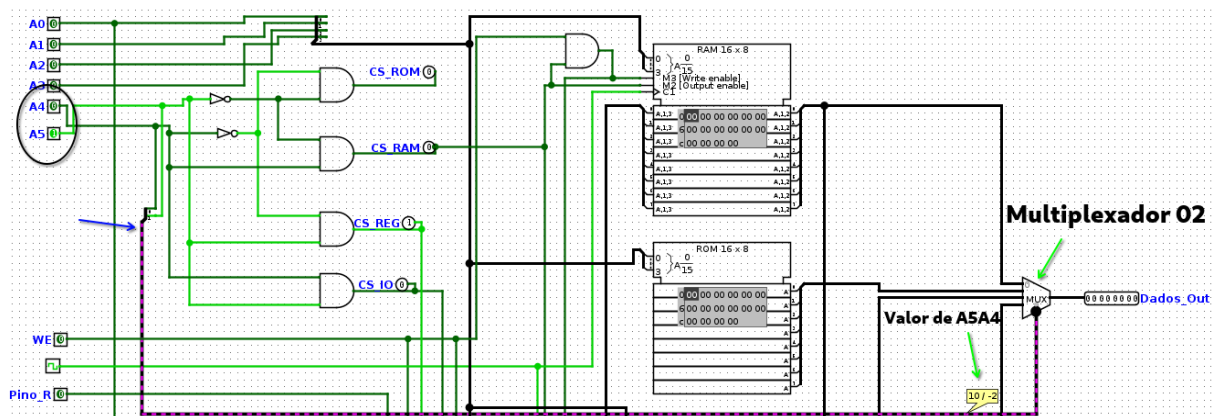


Figura 3 - Representação do comando A4, A5 no Multiplexador 02.

O bit de paridade ímpar de cada palavra gravada na RAM é gerado (na escrita) e recalculado (na leitura) pelo *GeradorParidade*, implementado como uma porta XOR de 8 entradas. O resultado é comparado com o bit armazenado numa RAM auxiliar de 1 bit por posição; a divergência entre os dois apaga o sinal Paridade_OK(resultado=0), indicando erro.

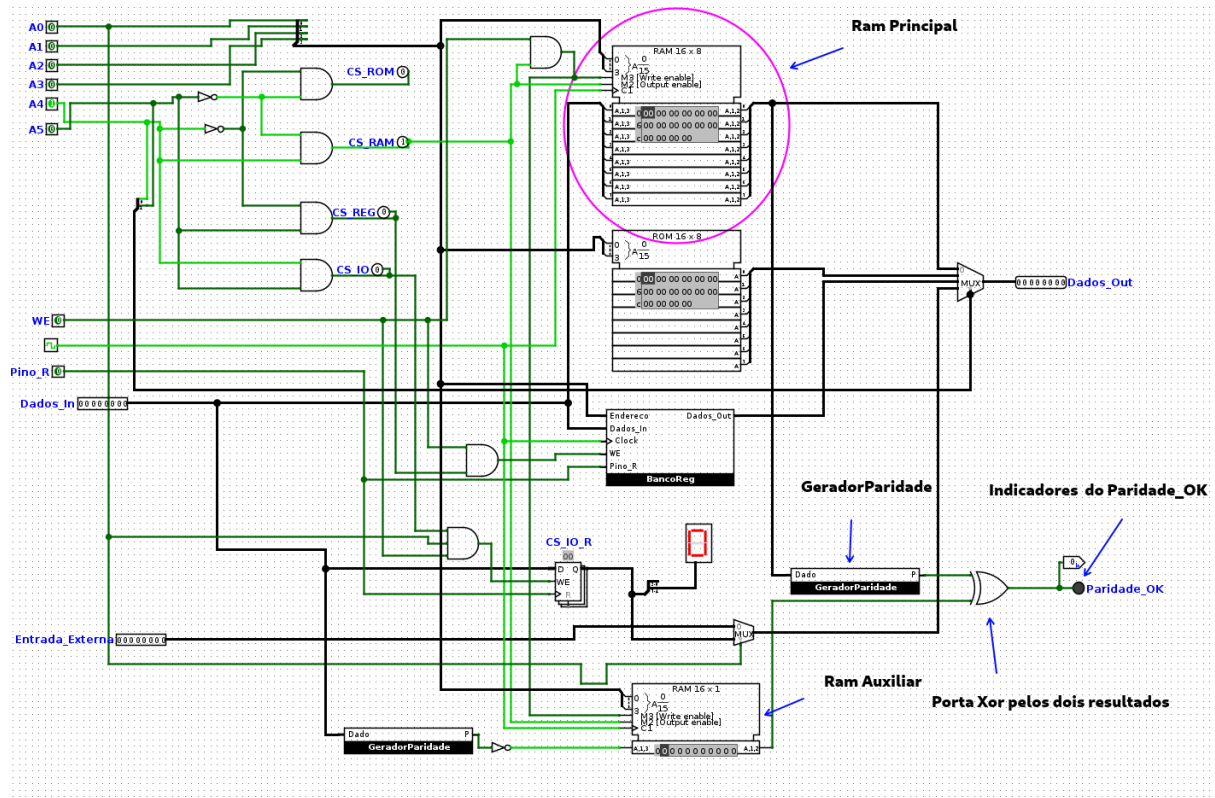


Figura 4 - Teste de paridade, juntamente com evidência dos componentes informados.

2.1 A cache: organização e divisão do endereço:

A cache implementada segue mapeamento direto, com 4 linhas, bloco de 2 bytes e metadado de 1 bit de validade + 3 bits de rótulo por linha. O mesmo endereço de 6 bits usado na memória é reinterpretado em três campos:

deslocamento	$\log_2(\text{tamanho do bloco})$	$\log_2(2) = 1 \text{ bit} \rightarrow A_0$
índice	$\log_2(\text{número de linhas})$	$\log_2(4) = 2 \text{ bits} \rightarrow A_2, A_1$
rótulo	$6 - 2 - 1$	3 bits $\rightarrow A_5, A_4, A_3$

Tabela 3 - Tabela do endereçamento da cache, de A0 a A5.

Exemplo demonstrativo (endereço 0x22 = 100010): deslocamento = 0, índice = 01, rótulo = 100. Esse endereço só pode ocupar a linha 1 da cache, e nela seria registrado o rótulo 100.

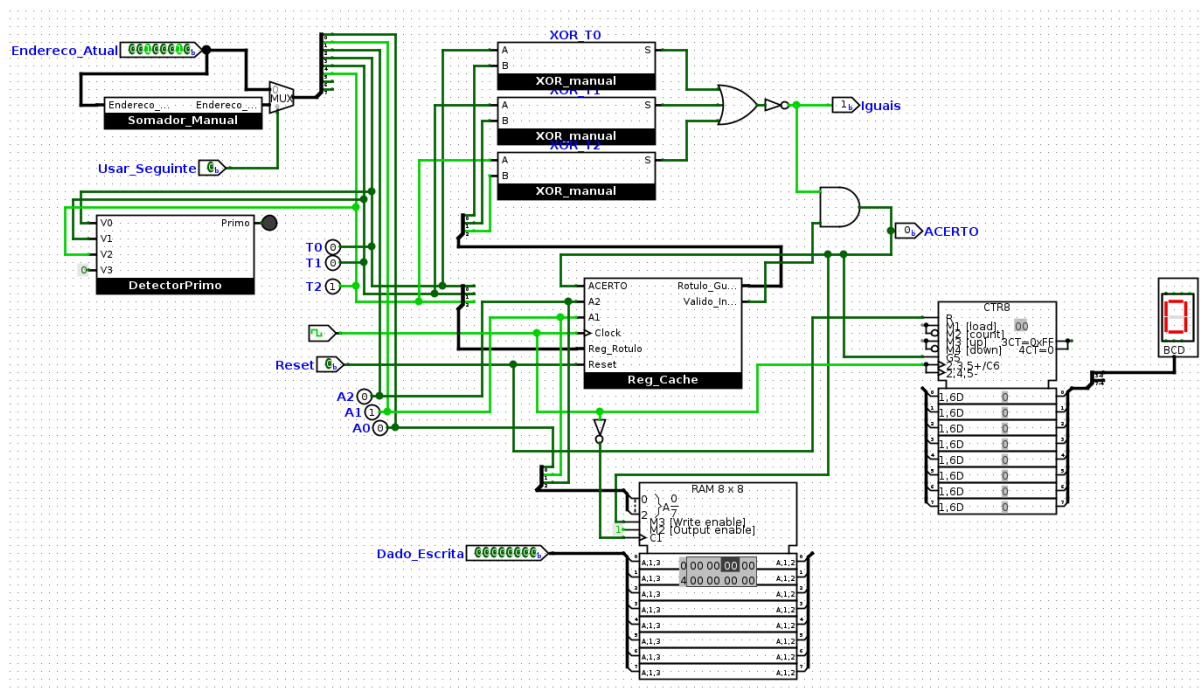


Figura 6 - Exemplo demonstrativo (endereço 0x22 = 100010).

O comparador dos rótulos (*XOR_manual*) foi construído a partir de portas AND, NOT e OR, seguindo o que foi pedido:

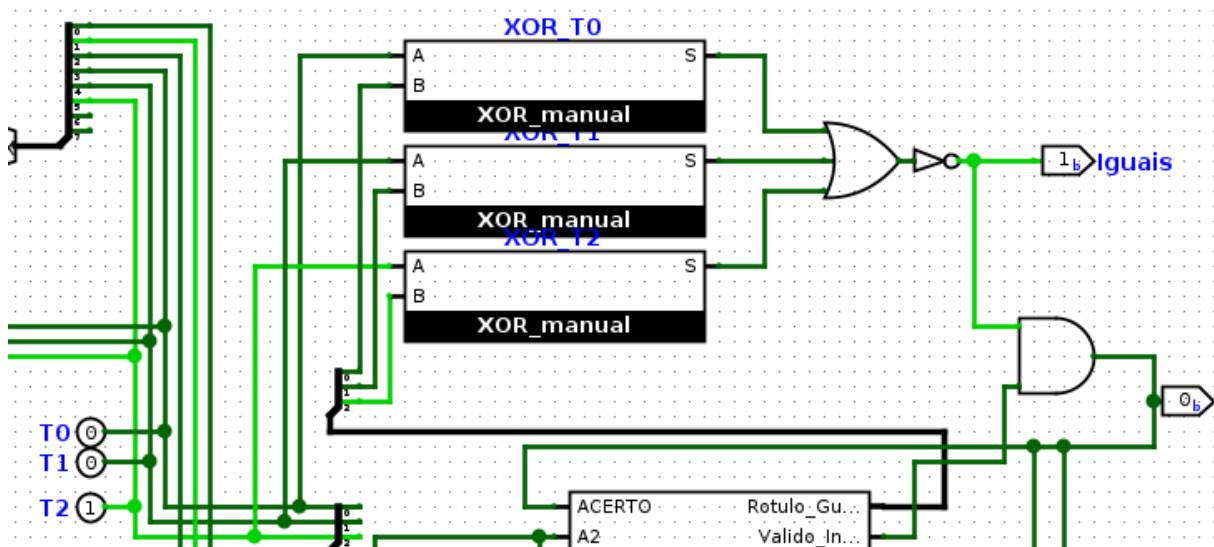


Figura 7 - Representação da formula iguais abaixo, no simulador.

- $\text{iguais} = \text{NOT}((T2 \oplus S2) \text{ OR } (T1 \oplus S1) \text{ OR } (T0 \oplus S0))$

Após ela:

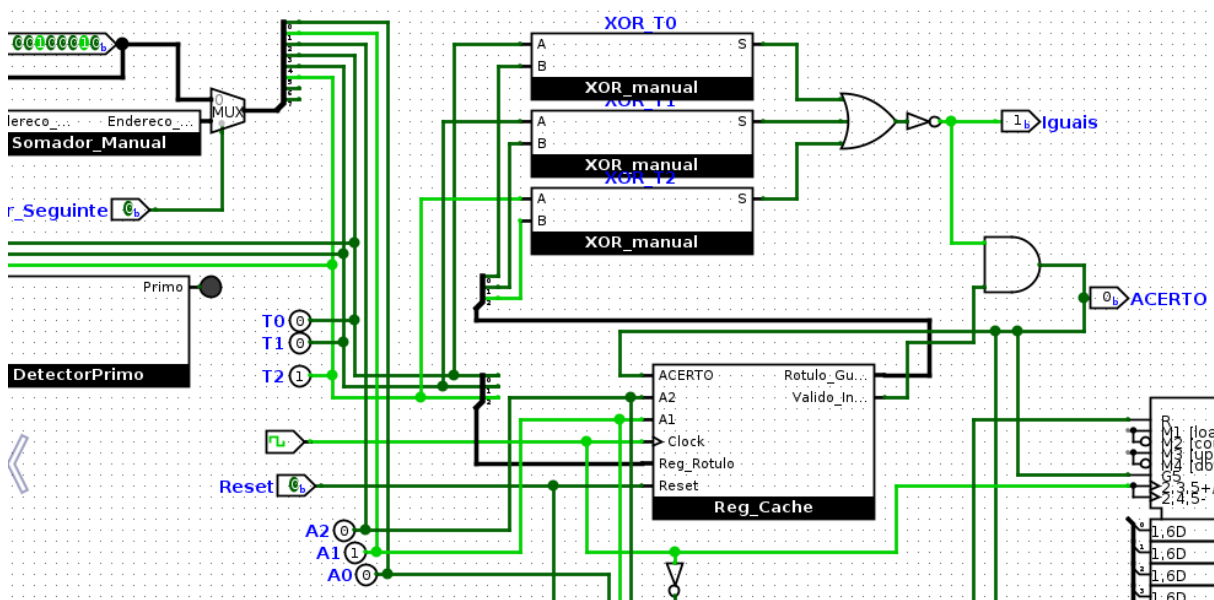


Figura 8 - Representação da formula do ACERTO abaixo, no simulador.

- $\text{ACERTO} = \text{iguais AND válido}[\text{índice}]$

Onde T2..T0 é o rótulo do endereço pedido e S2..S0 é o rótulo guardado na linha indicada pelo índice. Cada bloco XOR foi implementado como um subcircuito reutilizável (XOR_manual), montado com a fórmula:

$$S = (A \cdot \text{NOT } B) \text{ OR } (\text{NOT } A \cdot B).$$

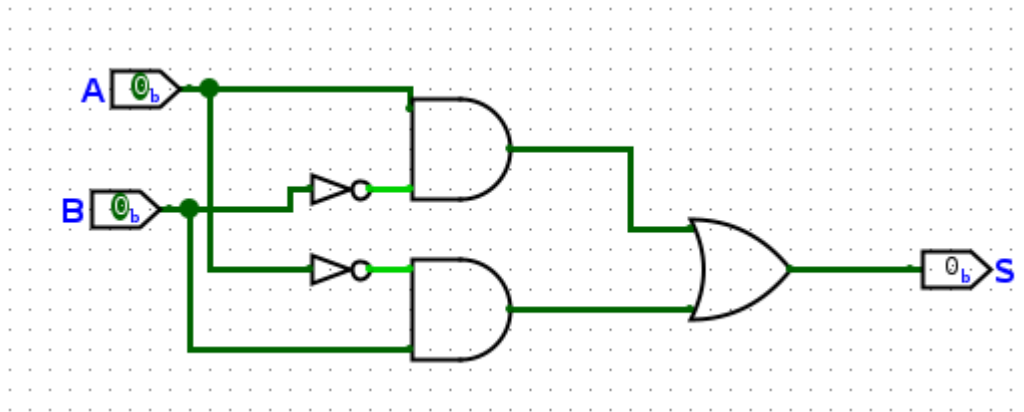


Figura 9 - Visão geral do XOR_manual.

O armazenamento das 4 linhas (rótulo + validade) foi implementado com registradores discretos, um par por linha, endereçados por um decodificador 2→4 (habilitado apenas quando há falta, via NOT(ACERTO)) e lidos de volta por dois multiplexadores 4×1 (um para o rótulo de 3 bits, outro para o bit de validade). Os bits de validade são zeráveis através de um pino de Reset comum às 4 linhas, atendendo ao critério de aceitação do entregável. Um contador de 8 bits, habilitado por ACERTO, alimenta um display de 7 segmentos com o total de acertos.

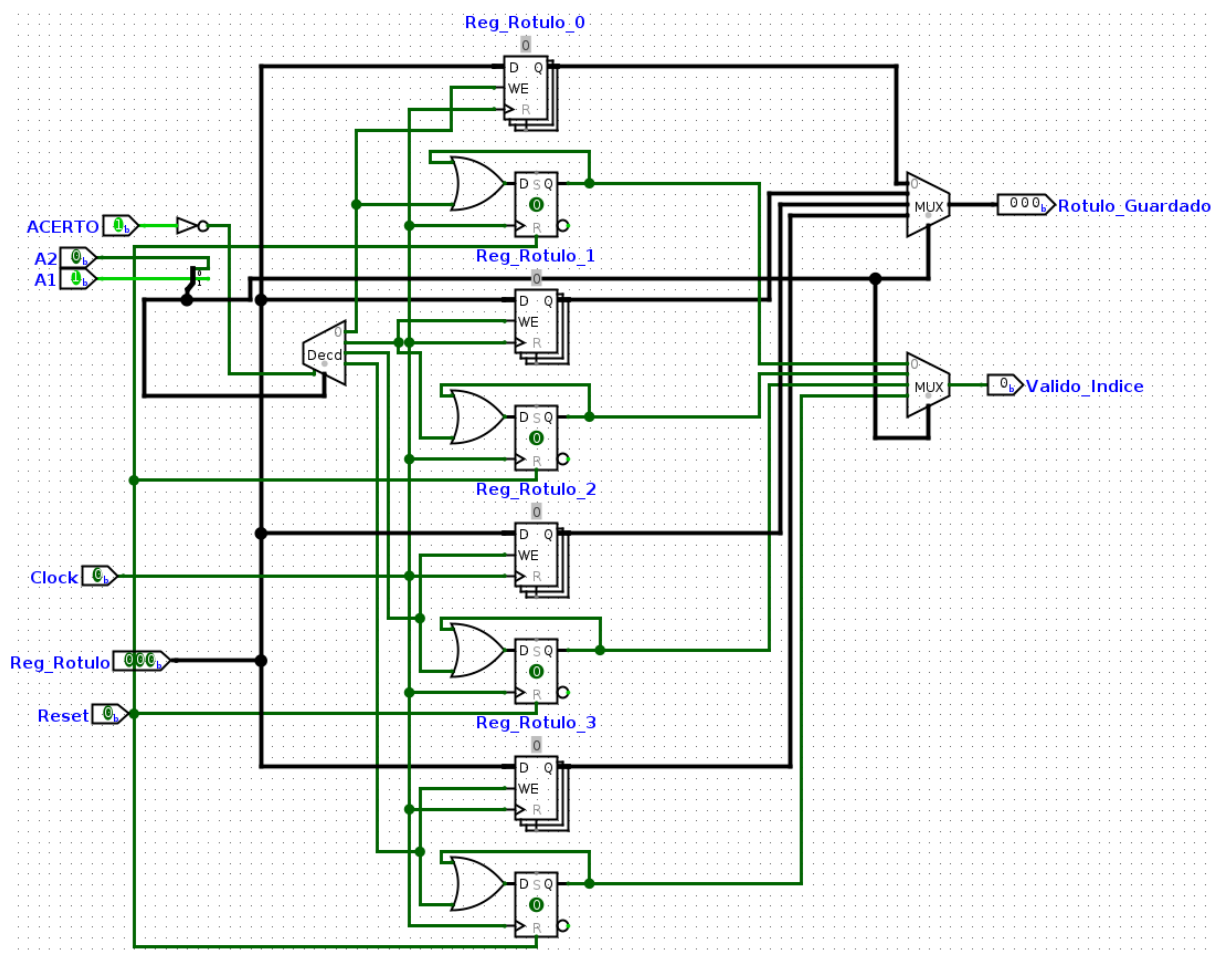


Figura 10 - Visão geral do Registrador da Cache (*Reg_Cache*).

2.2 Traço obrigatório de 16 acessos

O traço foi executado integralmente, resultando em 8 acertos e 8 faltas (taxa de 50% cada), sendo 4 faltas compulsórias e 4 faltas por conflito. A tabela completa, com todas as 11 colunas exigidas (as colunas endereço_bin, rotulo, indice, deslocamento, resultado, tipo_falta derivadas por fórmula a partir do endereço hexadecimal), está no arquivo parte1_planilha.xlsx.

- Indicadores calculados:

Acertos = 8 Faltas = 8 Taxa de acertos = 50%

Faltas compulsórias = 4 Faltas por conflito = 4

$AMAT = tempo_acerto + (taxa_faltas \times penalidade_falta)$

$AMAT = 1 + (0,5 \times 8) = 5 \text{ ciclos}$

2.3 Teste de conflito com o detector de números primos

Para evidenciar a principal limitação do mapeamento direto, foi construído o Componente 17 (detector de número primo de 4 bits), minimizado por mapa de Karnaugh a partir da tabela verdade dos primos entre 0 e 15 (2, 3, 5, 7, 11, 13):

Termo	Expressão	Números cobertos
T1	$v_3' \cdot v_2' \cdot v_1$	2, 3
T2	$v_3' \cdot v_1 \cdot v_0$	3, 7
T3	$v_2 \cdot v_1' \cdot v_0$	5, 13
T4	$v_2' \cdot v_1 \cdot v_0$	3, 11

Tabela 4 - Tabela dos primos entre 0 e 15.

Fórmula: $\text{primo} = (v_3' \cdot v_2' \cdot v_1) + (v_3' \cdot v_1 \cdot v_0) + (v_2 \cdot v_1' \cdot v_0) + (v_2' \cdot v_1 \cdot v_0)$

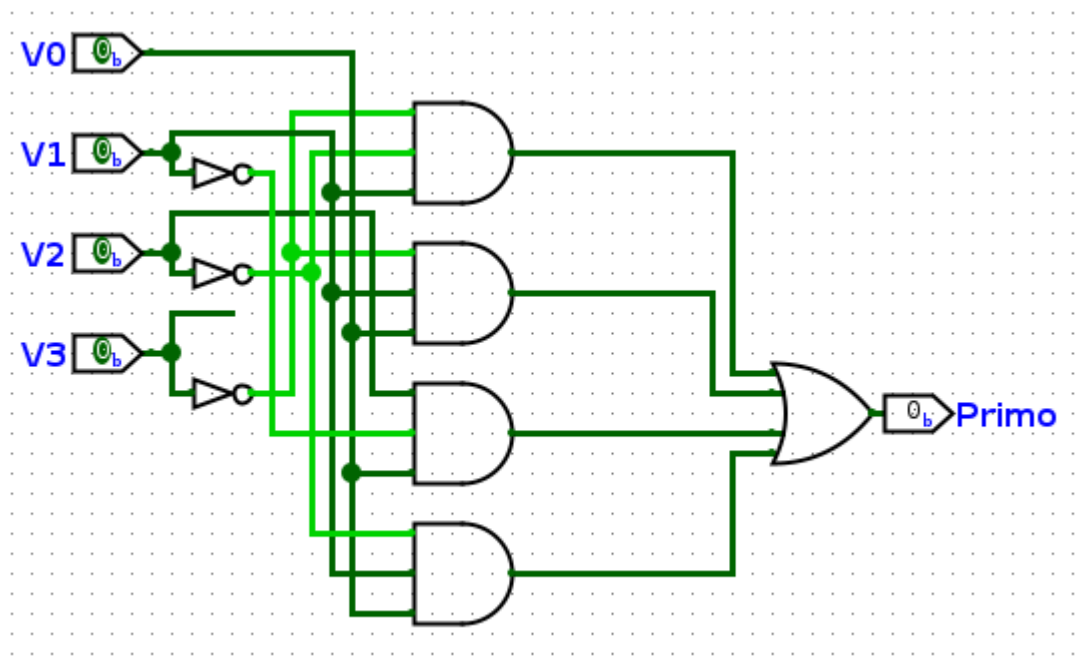


Figura 11 - Visão geral do Módulo de detecção de números primos (*DetectorPrimo*).

Percorrendo $v = 0$ a 7, o detector aponta corretamente $v = 2, 3, 5, 7$ como primos. Para cada um, foi gerado um acesso de leitura ao endereço v deslocado 3 posições à esquerda:

v	Endereço	Rótulo	Resultado obtido	Tipo
2	0x10	010	FALTA	compulsória
3	0x18	011	FALTA	conflito
5	0x28	101	FALTA	conflito
7	0x38	111	FALTA	conflito

Tabela 5 - Tabela dos acessos de leitura de números primos.

- Por que nenhum acerto ocorre, mesmo com 3 linhas ociosas:

Os quatro endereços gerados têm todos índice 00, porque v (3 bits, 0 a 7) desloca inteiramente para o campo de rótulo, deixando índice e deslocamento sempre zerados. Como a cache é de mapeamento direto, o índice determina de forma rígida qual linha um endereço pode ocupar. Os quatro rótulos diferentes disputam exclusivamente a linha 0, expulsando um ao outro a cada acesso, enquanto as linhas 1, 2 e 3 permanecem completamente ociosas.

Caso a cache tivesse 8 linhas em vez de 4: o índice passaria a ter 3 bits em vez de 2, tomados dos bits menos significativos do rótulo atual. Isso distribuiria os quatro valores de v (2, 3, 5, 7 ou em binário 010, 011, 101, 111) em índices diferentes entre si, eliminando o conflito neste caso específico e produzindo acertos nos acessos repetidos. O problema não desaparece de forma geral, mas ilustra como o número de linhas diretamente afeta a taxa de conflito de uma cache de mapeamento direto.

- Indicadores do teste de conflito:

Acertos = 0 Faltas = 4 Taxa de acertos = 0%

Faltas compulsórias = 1 Faltas por conflito = 3

AMAT = $1 + (1 \times 8) = 9$ ciclos

3. A Unidade de Controle Cabeada e a Microprogramada:

Foram projetadas dois estilos de UC (Unidade De Controle) encarregadas de planejar o fluxo dos dados e executar instruções no caminho de dados multiciclo de 8 bits:

- A Unidade de Controle Cabeada (utilizando-se portas lógicas e Flip-Flops)
- A Unidade de Controle Microprogramada (utilizando-se memória ROM de microcódigo e microcontador).

3.1. Arquitetura da Unidade de Controle Cabeada:

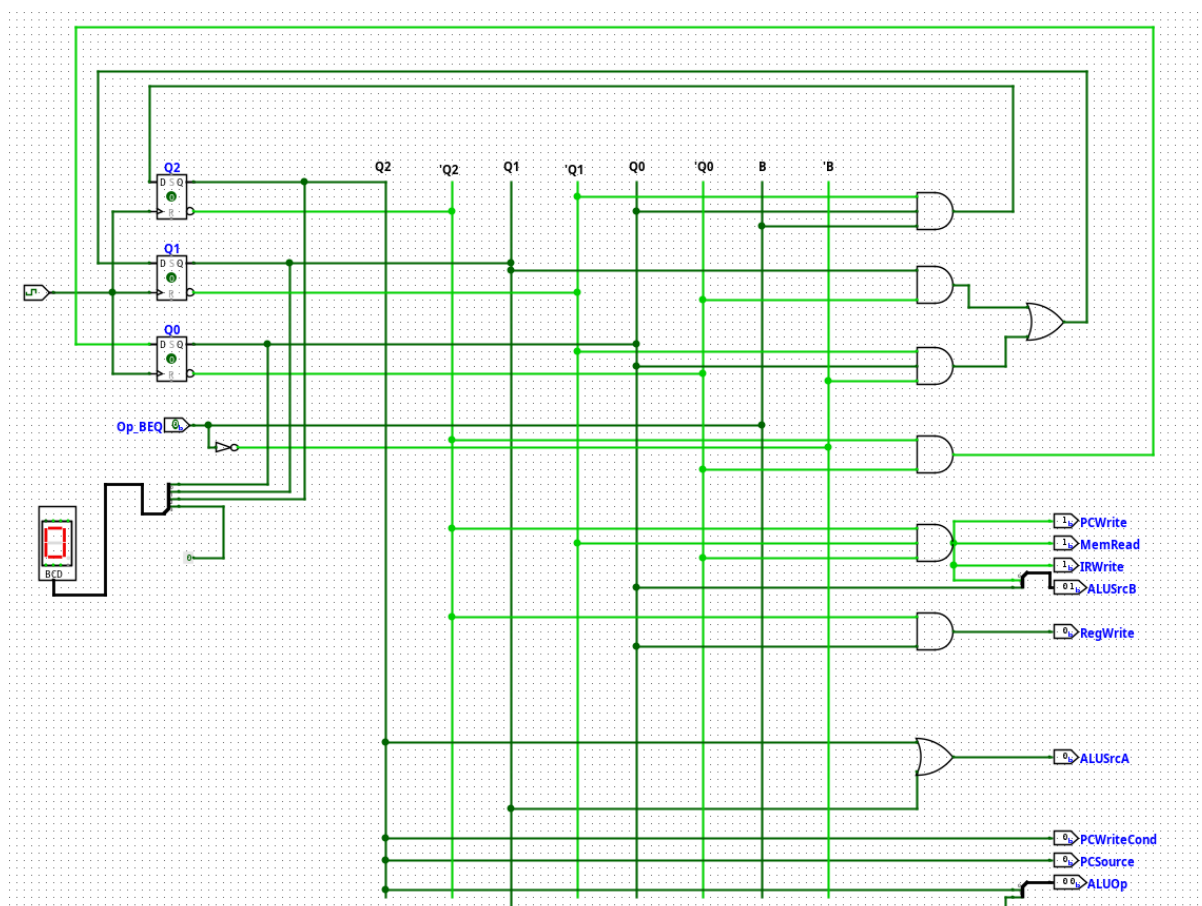


Figura 12 - Visão Geral da Unidade de Controle Microcabeada.

Essa é uma versão sintetizada da máquina de estados finitos (FSM) com base em lógica combinacional e fazendo o uso de portas AND, OR e NOT com elementos de memória sequencial (3 Flip-Flops Tipo D). Como a máquina possui exatamente 5

estados ativos (S0 a S4), são necessário $\lceil \log_2(5) \rceil = 3$ bits para representar o estado atual (Q2,Q1,Q0).

Os estados e suas ações no caminho de dados são:

- S0 (000 - Busca): Incremento ao contador do programa ($PC \leftarrow PC + 4$) e carrega a instrução da memória ROM para o registrador de instrução ($IR \leftarrow Mem[PC]$).
- S1 (001 - Decodificação): Lê dois registradores rs e rt para os registradores temporários A e B, e realiza um pré-cálculo do endereço de desvio em $ULAOut \leftarrow PC + (ext(imm) \ll 2)$.
- S2 (010 - Execução Tipo-R): Executa a operação lógica/aritmética na ULA entre os registradores A e B ($ULAOut \leftarrow A \text{ op } B$).
- S3 (011 - Conclusão Tipo-R): Escreve o resultado armazenado em ULAOut de volta no banco de registradores ($Reg[rd] \leftarrow ULAOut$).
- S4 (100 - Conclusão de Desvio): Calcula $A - B$ na ULA; se a flag Zero for ativada atualiza o PC com o endereço de desvio em ULAOut ($PC \leftarrow ULAOut$).

As combinações restantes que não foram utilizadas (101,110,111) foram tratadas como condições de segurança, forçando o retorno da máquina ao estado de inicial de busca S0(000) para evitar possíveis travamentos.

3.2. Tabela Verdade e Tabela Transicional dos Estados:

Estado	Código	PCWrite	MemRead	IRWrite	ALUSrc B	RegWrite	ALUSrc A	PCWriteCond	PCSource	ALUOp
S0	000	1	1	1	01	0	0	0	0	00
S1	001	0	0	0	10	1	0	0	0	00
S2	010	0	0	0	00	0	1	0	0	10
S3	011	0	0	0	10	1	1	0	0	10
S4	100	0	0	0	00	0	1	1	1	01

Tabela 6 - Tabela Verdade

Estado (Q2Q1Q0)	Código	Entrada (B)	Próximo Estado(D2,D1, D0)	PC Write	Mem Read	IRWrite	ALUSr cB	RegWrite	ALU SrcA	PCWrit eCond	PCSou rce	ALUOp
S0	000	X	001	1	1	1	01	0	0	0	0	00
S1	001	0 1	010 100	0	0	0	10	1	0	0	0	00
S2	010	X	011	0	0	0	00	0	1	0	0	10
S3	011	X	000	0	0	0	10	1	1	0	0	10
S4	100	X	000	0	0	0	00	0	1	1	1	01

Tabela 7 - Tabela Transicional de Estados.

A bifurcação entre os estados ocorre no estado S1, onde o bit de opcode da instrução determina o próximo passo: instruções do tipo-R (Opcode = 0) e avançam o estado S2, enquanto instruções de desvio beq(OpCode = 1) seguem ao estado S4.

3.3. Minimização por meio de Mapas de Karnaugh e Equações Booleanas:

Q2	Q1	Q0	D	T
0	0	0	0	0
1	0	1	1	0
2	1	0	2	1
3	1	0	3	1
4	1	0	4	X
5	0	1	5	X
6	0	1	6	X
7	1	0	7	X

Q0/Q2	00	01	11	10
00	1	X	1	1

10	X	X	1	1
----	---	---	---	---

Tabela 8 - Mapa de Karnaugh para Próximo Estado D2

Equação Booleana: $D_2 = \overline{Q_1} \cdot Q_0 \cdot B$

Q2	Q1	Q0	D	T
0	0	0	0	0
1	0	1	1	1
2	1	0	2	1
3	1	0	3	0
4	1	0	4	X
5	0	1	5	X
6	0	1	6	X
7	1	0	7	X

Q1/Q2	00	01	11	10
00	1	1	1	0
01	1	1	1	1

Tabela 8 - Mapa de Karnaugh para D1 = Q1 + Q2' * Q0

Equação Booleana: $D_1 = (Q_1 \cdot \overline{Q_0}) + (\overline{Q_1} \cdot Q_0 \cdot \overline{B})$

Q2	Q1	Q0	D	T
0	0	0	0	0
1	0	1	1	1
2	1	0	2	2
3	1	0	3	2
4	1	0	4	X
5	0	1	5	X
6	0	1	6	X

7	1	0	7	X
---	---	---	---	---

Q0/Q2	00	01	11	10
00	0	X	1	0
10	X	X	1	0

Tabela 9 - Mapa de Karnaugh para $D_0 = Q_2' \cdot Q_0'$

Equação Booleana: $D_0 = \overline{Q_2} \cdot \overline{Q_0}$

As demais equações dos Sinais de Controle são:

- **PCWrite** = $\overline{Q_2} \cdot \overline{Q_1} \cdot \overline{Q_0}$
- **MemRead** = $\overline{Q_2} \cdot \overline{Q_1} \cdot \overline{Q_0}$
- **IRWrite** = $\overline{Q_2} \cdot \overline{Q_1} \cdot \overline{Q_0}$
- **ALUSrcB (Bit 1)** = Q_0
- **ALUSrcB (Bit 0)** = $\overline{Q_2} \cdot \overline{Q_1} \cdot \overline{Q_0}$
- **RegWrite** = $\overline{Q_2} \cdot Q_0$
- **ALUSrcA** = $Q_2 + Q_1$
- **PCWriteCond** = Q_2
- **PCSource** = Q_2
- **ALUOp (Bit 1)** = Q_1
- **ALUOp (Bit 0)** = Q_2

3.4 Sinais de Controle, Tabela de Tempo e Desempenho (CPI)

A unidade de controle gera 9 sinais de controle que totalizam 11 bits no total, que tem a função de comandar os multiplexadores, habilitações de escrita e operações da ULA.

Através da decodificação direta dos estados (S0 a S4), obtêm-se equações minimizadas para os sinais:

- $PCWrite = S_0 = \overline{Q_2} \cdot \overline{Q_1} \cdot \overline{Q_0}$
- $PCWriteCond = S_4 = Q_2 \cdot \overline{Q_1} \cdot \overline{Q_0}$
- $PCSource = S_4 = Q_2 \cdot \overline{Q_1} \cdot \overline{Q_0}$
- $MemRead = S_0 = \overline{Q_2} \cdot \overline{Q_1} \cdot \overline{Q_0}$

- $IRWrite = S_0 = \overline{Q_2} \cdot \overline{Q_1} \cdot \overline{Q_0}$
- $RegWrite = S_3 = \overline{Q_2} \cdot Q_1 \cdot Q_0$
- $ALUSrcA = S_2 + S_4 = (\overline{Q_2} \cdot Q_1 \cdot \overline{Q_0}) + (Q_2 \cdot \overline{Q_1} \cdot \overline{Q_0})$
- $ALUSrcB_1 = S_1 = \overline{Q_2} \cdot \overline{Q_1} \cdot Q_0$
- $ALUSrcB_0 = S_0 = \overline{Q_2} \cdot \overline{Q_1} \cdot \overline{Q_0}$
- $ALUOp_1 = S_2 = \overline{Q_2} \cdot Q_1 \cdot \overline{Q_0}$
- $ALUOp_0 = S_4 = Q_2 \cdot \overline{Q_1} \cdot \overline{Q_0}$

Quanto a CPI, considerando as instruções:

1. Tipo-R (add, sub, and, or) que percorre 4 ciclos dos estados $S_0 \rightarrow S_1 \rightarrow S_2 \rightarrow S_3$
2. Tipo BEQ que percorre 3 ciclos dos estados $S_0 \rightarrow S_1 \rightarrow S_2 \rightarrow S_3$

O CPI médio fica:

$$CPI_{médio} = (0,70 \times 4) + (0,30 \times 3) = 2,8 + 0,9 = 3,7 \text{ ciclos por instrução}$$

3.5 Unidade de Controle Microprogramada:

A implementação microprogramada faz a substituição da sequência de portas lógicas por uma memória ROM de microcódigo de 8 posições por 8 bits e um microcontador de programa (PC) de 3 bits.

A palavra de controle de 8 bits está organizada em 4 campos de 2 bits:

1. **ALUOP [Bits 7-6]:** 00 = Soma; 01 = Subtração; 10 = Definido por funct; 11 = Reservado.
2. **Fonte [Bits 5-4]:** 00 = PC + Constante 4; 01 = RegA e RegB; 10 = PC + ext(imm) < 2.
3. **Ação [Bits 3-2]:** 00 = Nenhuma; 01 = $IR \leftarrow Mem[PC]$; 10 = $Reg[rd] \leftarrow ULAOut$; 11 = $PC \leftarrow ULAOut$ se Zero.
4. **Sequenciamento [Bits 1-0]:** 00 = Próxima microinstrução ($uPC+1$); 01 = Volta à busca ($uPC \leftarrow 0$); 10 = Despacho pelo Opcode; 11 = Reservado.

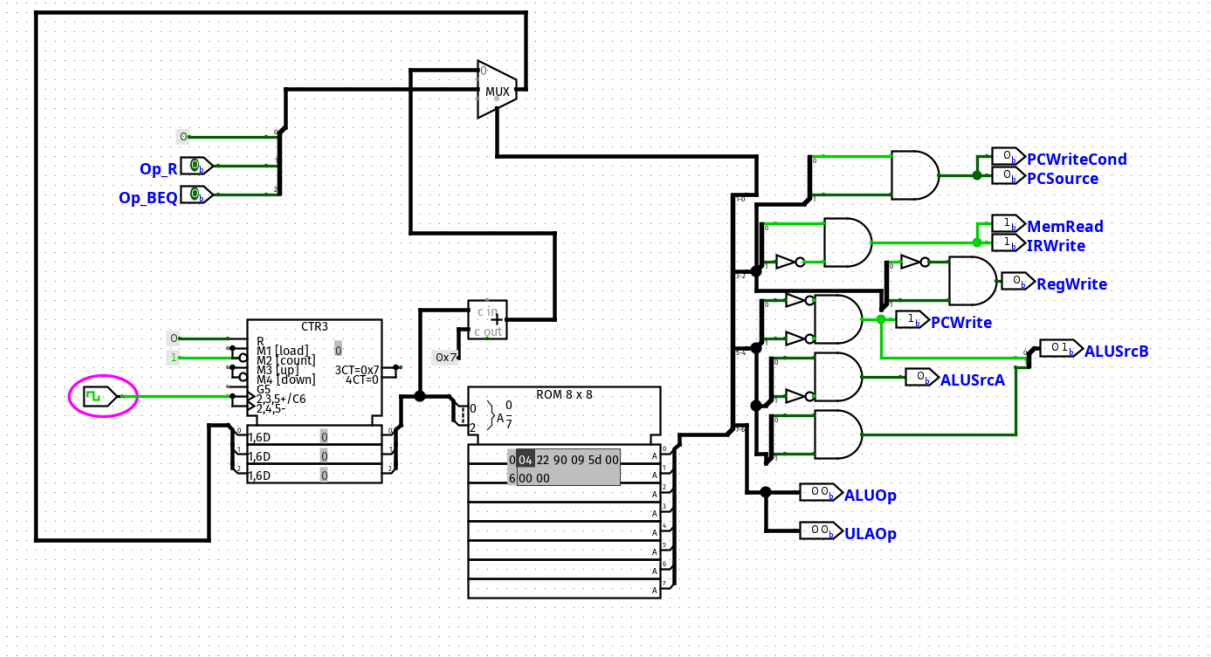


Diagrama de Estados Completo da Unidade de Controle MIPS

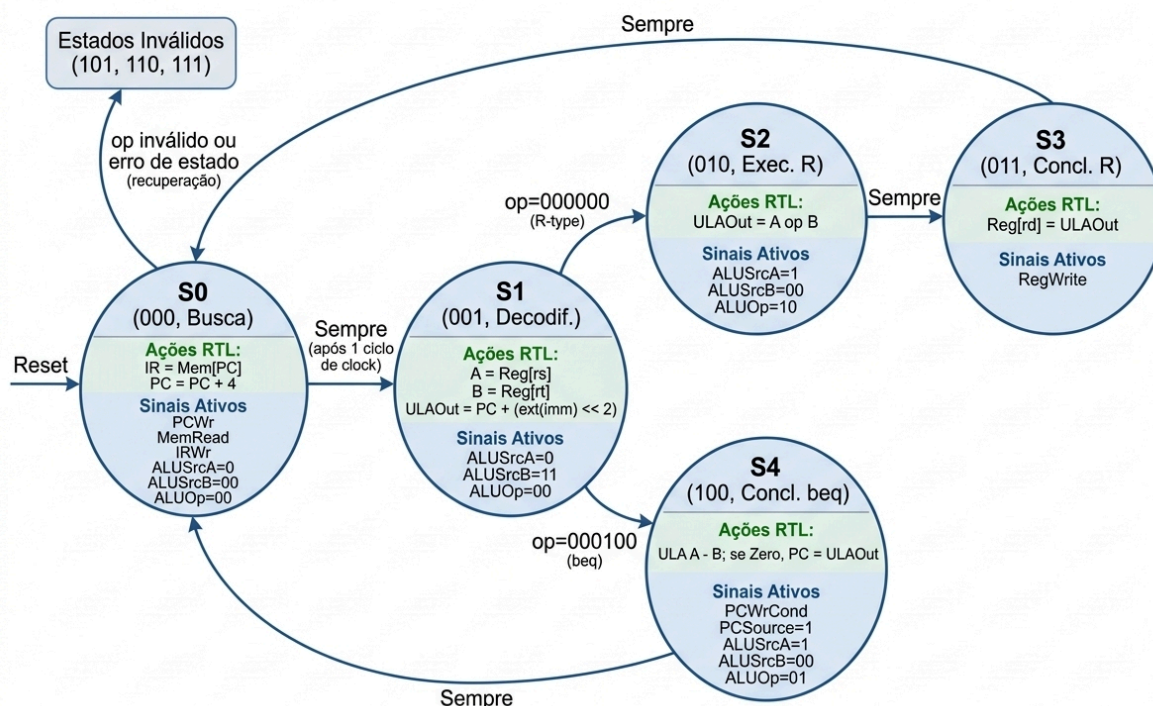


Figura 14 - Diagrama de Estados da Unidade de Controle MIPS

3.6. Comparação entre as duas versões:

Ambas versões apresentaram pontos fortes e fracos, com uma se provando ser preferível em certas tarefas do que outras, se destacaram as seguintes diferenças quanto ao desempenho e os custos de hardware:

1. Consumo de Recursos de Hardware:

A versão cabeada consumiu em total 9 portas lógicas (4 NOTS, 8 ANDs e 2 ORs) somado aos 3 Flip-Flops de tipo D. Em contrapartida, a versão microprogramada consumiu 64 bits de memória ROM, além de uPC de 3 bits e do MUX de despacho.

2. Facilidade:

A versão cabeada foi substancialmente mais difícil de ser projetada, com várias tentativas e erros sendo feitos durante o trajeto devido a conflitos entre fios, análise da lógica empregada pelas portas lógicas e traçamento de caminho. Por outro lado a cabeada provou-se muitos mais simples e intuitiva de projetar devido ao sequenciamento sendo resolvido por meio da estruturação de campos de bits na tabela do microprograma e a gravação do microcodigo.txt.

3. Extensibilidade e Manutenibilidade:

Todas funções de próximo estado da versão cabeada (D2,D1,D0) e vários sinais de controles iriam ter que ser recalculados caso houvesse a necessidade de adicionar uma nova operação, refazendo os mapas de Karnaugh por completo e a fiação das portas. Na microprogramada, a resolução é mais simples, apenas precisando programar uma ou duas microinstruções nas posições restantes da ROM (0x5 e 0x6) e atualizar a lógica de despacho, sem maiores alterações no circuito físico.

4. Aplicação em Projetos Reais:

Em arquiteturas RISC como MIPS e ARM nativos, a abordagem cabeada é mais vantajosa devido à velocidade máxima de clock e a baixa latência por ciclo, que justifica a complexidade do circuito em si. Já a microprogramada é melhor para estruturas CISC como as x86, devido a fato de lidar com instruções complexas e a flexibilidade para corrigir bugs ou atualizações no microcódigo se sobressai à velocidade do ciclo de clock.

6. REFLEXÃO

6.1 Ao analisar o arquivo .wat gerado para a função soma , descreva o fluxo das instruções em comparação com o assembly MIPS (add / addu). Como o WASM manipula as variáveis locais sem usar registradores como \$t0 ou \$t1 ?

6.2 No MIPS, um overflow na instrução add gera uma exceção em hardware, enquanto addu ignora silenciosamente. Como a sua função soma_segura simula o comportamento da arquitetura? Foi mais custoso tratar via software (WASM/C)?

6.3 Explique os valores do contraexemplo gerado pelo Map2Check. Como a representação em complemento de dois explica a falha identificada pela ferramenta?

7. CONCLUSÃO

8. Declaração de uso de ferramentas de IA generativa

Ferramentas de IA generativa foram usadas ao longo de toda a Parte I e Parte II como ajudante de construção nos passo a passo no Logisim-Evolution. A IA não montou nenhum circuito; cada componente, conexão e teste foi executado manualmente pelos integrantes, com a IA explicando os conceitos, auxiliando em possíveis passos conceituais e revisando o circuito em casos de apontamento de

erros de conexão. A IA também auxiliou nos conceitos das formulações das fórmulas da planilha eletrônica.

9. REFERÊNCIAS

TANENBAUM, A. S. Organização Estruturada de Computadores. 6ª ed. São Paulo: Pearson, 2013. Cap. 2, p. 61-67 (códigos de detecção de erro / paridade); Cap. 4, p. 243-244 (memória cache, mapeamento direto).

PATTERSON, D. A.; HENNESSY, J. L. Organização e Projeto de Computadores: a Interface Hardware/Software.

Cap. 5 — A hierarquia de memória (cache diretamente mapeada; tempo médio de acesso à memória).